Phân tích yêu cầu và giải pháp Agent

https://gemini.google.com/app/44db9c3e171cfabc

User prompt: Dựa theo file này phân tích yêu cầu và đề xuất cách giải?

Response: Dựa trên tài liệu **AI Challenge 11 — Claim Assessment AI Agent**, dưới đây là phần phân tích chi tiết các yêu cầu cốt lõi và đề xuất phương án giải quyết toàn diện để hoàn thành thử thách trong khoảng thời gian 6–8 giờ.

1. Phân tích yêu cầu (Requirements Analysis)

A. Mục tiêu tổng quan

Xây dựng một **Conversational AI Agent** tự động hóa quá trình đánh giá hồ sơ bồi thường bảo hiểm (Claim Assessment) nhằm rút ngắn thời gian xử lý từ vài ngày xuống còn vài phút.

B. Thành phần kỹ thuật chính

1. **Hệ thống 4 công cụ (Tools)** bắt buộc phải được lập trình dưới dạng hàm để Agent gọi:
    - lookupPolicy(policyId): Trả về điều khoản hợp đồng (quyền lợi, hạn mức, điều khoản loại trừ, đồng chi trả, thời gian chờ).
    - calculateBenefit(policyId, claimType, amount): Tính toán số tiền được chi trả, mức đồng chi trả (copay) và hạn mức còn lại.
    - verifyDocument(documentId): Kiểm tra loại tài liệu, trạng thái hoàn thiện và các lỗi phát sinh.
    - checkMedicalNecessity(diagnosis, procedures): Kiểm tra tính phù hợp về mặt y tế giữa chẩn đoán và phương pháp điều trị.
2. **Quy tắc hành vi của Agent (Agent Behavior)**:
    - Phải gọi công cụ theo đúng **trình tự logic**: Xác minh tài liệu → Tra cứu chính sách → Kiểm tra sự cần thiết y tế → Tính toán quyền lợi.
    - Kiểm tra toàn bộ tài liệu nộp, không được bỏ sót bất kỳ tài liệu nào.
    - Xác thực thời hạn hiệu lực của yêu cầu bồi thường so với thời gian của hợp đồng.
    - **Quy tắc quan trọng**: Nếu thiếu tài liệu bắt buộc, Agent **phải đề xuất yêu cầu bổ sung thông tin (request_more_info)**, tuyệt đối không được từ chối (reject).
3. **Báo cáo đánh giá (Assessment Report)**: Báo cáo đầu ra cần cấu trúc rõ ràng với 6 phần:
    - *Document Review* (Đánh giá tài liệu)
    - *Policy Verification* (Xác thực chính sách)
    - *Medical Necessity* (Sự cần thiết y tế)
    - *Benefit Calculation* (Tính toán quyền lợi)
    - *Recommendation* (Kiến nghị: APPROVE, REJECT, hoặc REQUEST_MORE_INFO)
    - *Policy Citations* (Trích dẫn cụ thể điều khoản chính sách cho từng nhận định)
4. **Ràng buộc (Constraints)**:
    - Không được tự bịa đặt điều khoản (hallucinate) mà bắt buộc phải thông qua lookupPolicy.

- Không được phê duyệt số tiền vượt quá hạn mức quyền lợi.
- Xử lý chính xác các trường hợp tài liệu nộp lên không đúng chủng loại yêu cầu.

2. Đề xuất cách giải (Solution Proposal)

Bước 1: Lựa chọn công nghệ & Kiến trúc hệ thống

- **Ngôn ngữ lập trình**: Python.
- **LLM Model**: Sử dụng **Claude 3.5 Sonnet** hoặc **GPT-4o** (thông qua API) nhờ khả năng xử lý Function Calling và tuân thủ system prompt rất tốt.
- **Agent Framework**: Sử dụng **LangGraph** hoặc xây dựng vòng lặp Function Calling thuần túy bằng Python (OpenAI/Anthropic SDK) để kiểm soát chặt chẽ thứ tự gọi công cụ. LangGraph rất phù hợp để ép buộc luồng tuần tự (Document Verification → Policy Lookup → Medical Necessity → Benefit Calculation).
- **Database giả lập**: Dùng một file JSON hoặc SQLite lưu trữ dữ liệu mẫu cho các chính sách (policies), hồ sơ claim và trạng thái tài liệu để các công cụ truy vấn.

Bước 2: Thiết kế 4 Test Cases (Theo yêu cầu Setup)

Ta cần chuẩn bị 3 bộ dữ liệu test (dạng JSON hoặc Python dictionaries) bao gồm đầy đủ thông tin chi tiết về Claim, Policy, Document List và Expected Outcome:

1. **Case 1 (Phê duyệt trực tiếp - Straightforward Approval)**:
   - Hồ sơ đầy đủ giấy tờ, đúng hạn, đúng danh mục bệnh được chi trả, số tiền nằm trong hạn mức.
   - *Expected Outcome*: APPROVE.
2. **Case 2 (Từ chối - Rejection)**:
   - Hồ sơ đầy đủ nhưng thuộc điều khoản loại trừ trong hợp đồng hoặc vượt quá hạn mức tối đa.
   - *Expected Outcome*: REJECT.
3. **Case 3 (Yêu cầu bổ sung - Request More Info)**:
   - Hồ sơ thiếu hóa đơn đỏ hoặc giấy ra viện bắt buộc.
   - *Expected Outcome*: REQUEST_MORE_INFO (kèm chỉ định rõ tài liệu còn thiếu).

Bước 3: Viết System Prompt cho Agent

System prompt cần quy định rõ ràng các ràng buộc đạo đức và nghiệp vụ:

"Bạn là một chuyên gia đánh giá bồi thường bảo hiểm y tế tự động. Bạn phải luôn gọi các công cụ theo đúng thứ tự: verifyDocument → lookupPolicy → checkMedicalNecessity → calculateBenefit. Bạn tuyệt đối không được tự bịa đặt điều khoản bảo hiểm. Nếu phát hiện thiếu tài liệu, bạn phải đề xuất 'REQUEST_MORE_INFO'. Mọi kết luận trong báo cáo phải trích dẫn chính xác mã điều khoản lấy từ công cụ."

Bước 4: Lên kế hoạch thực hiện (Timeline dự kiến: 6–8 giờ)

Thời gian	Công việc cụ thể
Giờ 1–2	• Thiết lập cấu trúc project Python. • Viết dữ liệu giả lập (Mock Database) cho 3 Test Cases. • Hiện thực hóa 4 hàm công cụ (lookupPolicy, calculateBenefit, verifyDocument, checkMedicalNecessity).
Giờ 3–4	• Tích hợp LLM API và cấu hình Function Calling. • Xây dựng luồng logic của Agent (đảm bảo gọi đúng thứ tự và ghi lại log toàn bộ tool calls).
Giờ 5–6	• Viết hàm sinh báo cáo tự động (Assessment Report) theo đúng 6 phần chuẩn. • Chạy thử nghiệm với 3 Test Cases và tinh chỉnh System Prompt nếu có hiện tượng hallucination hoặc bỏ sót bước.
Giờ 7–8	• Hoàn thiện viết tài liệu (Writeup) giải thích thiết kế công cụ và System Prompt. • Đóng gói mã nguồn, log kết quả chạy và đẩy lên GitHub repository theo đúng yêu cầu nộp bài.

Bạn có muốn tôi hỗ trợ viết mã nguồn chi tiết cho phần **4 công cụ (Tools)** hoặc **System Prompt** để bắt đầu

triển khai ngay không?

User prompt: Vâng

Response: Dưới đây là mã nguồn Python mẫu bao gồm cơ sở dữ liệu giả lập (Mock Database), định nghĩa **4 công cụ (Tools)** theo yêu cầu của đề bài, và **System Prompt** cho AI Agent.

1. Cơ sở dữ liệu giả lập (mock_database.py)

File này lưu trữ thông tin chính sách bảo hiểm, tài liệu nộp và các quy tắc y tế để các công cụ truy vấn.

```
# Mock Database cho Claim Assessment Agent

POLICIES_DB = {
    "POL-001": {
        "policyId": "POL-001",
        "holderName": "Nguyen Van A",
        "status": "ACTIVE",
        "effectiveDate": "2025-01-01",
        "expiryDate": "2026-12-31",
        "benefits": {
            "Inpatient": {"limit": 50000000, "copay": 0.1, "waitingPeriodDays":
30},
            "Outpatient": {"limit": 10000000, "copay": 0.2,
"waitingPeriodDays": 0}
        },
        "exclusions": ["Cosmetic surgery", "Pre-existing conditions within 12
months"]
    },
    "POL-002": {
        "policyId": "POL-002",
        "holderName": "Tran Thi B",
        "status": "ACTIVE",
        "effectiveDate": "2023-06-01",
        "expiryDate": "2024-06-01", # Đã hết hạn (hết hiệu lực)
        "benefits": {
            "Inpatient": {"limit": 30000000, "copay": 0.1, "waitingPeriodDays":
30}
        },
        "exclusions": ["Experimental treatments"]
    }
}

DOCUMENTS_DB = {
    "DOC-101": {
        "documentId": "DOC-101",
        "type": "Medical Bill / Invoice",
        "status": "complete",
        "issues": []
```

```
    },
    "DOC-102": {
        "documentId": "DOC-102",
        "type": "Discharge Summary",
        "status": "incomplete",
        "issues": ["Missing physician signature and official hospital stamp"]
    },
    "DOC-103": {
        "documentId": "DOC-103",
        "type": "Prescription",
        "status": "complete",
        "issues": []
    }
}

MEDICAL_RULES_DB = {
    ("Acute Appendicitis", "Appendectomy"): True,
    ("Common Cold", "Inpatient Surgery"): False,
    ("Fractured Arm", "X-Ray and Casting"): True
}
```

2. Định nghĩa 4 Công cụ (agent_tools.py)

Các hàm công cụ được triển khai dưới dạng Python functions để LLM có thể gọi thông qua cơ chế Function Calling.

```
import json
from mock_database import POLICIES_DB, DOCUMENTS_DB, MEDICAL_RULES_DB

def lookupPolicy(policyId: str) -> str:
    """Trả về điều khoản hợp đồng: quyền lợi, hạn mức, điều khoản loại trừ,
đồng chi trả, thời gian chờ."""
    policy = POLICIES_DB.get(policyId)
    if not policy:
        return json.dumps({"error": f"Policy ID {policyId} not found."})
    return json.dumps(policy, ensure_ascii=False)

def calculateBenefit(policyId: str, claimType: str, amount: float) -> str:
    """Tính toán số tiền được chi trả, mức đồng chi trả (copay) và hạn mức còn
lại."""
    policy = POLICIES_DB.get(policyId)
    if not policy:
        return json.dumps({"error": f"Policy ID {policyId} not found."})

    benefit_info = policy["benefits"].get(claimType)
    if not benefit_info:
        return json.dumps({"error": f"Claim type {claimType} not covered under
this policy."})

    limit = benefit_info["limit"]
```

```
    copay_rate = benefit_info["copay"]

    if amount > limit:
        covered_before_copay = limit
    else:
        covered_before_copay = amount

    copay_amount = covered_before_copay * copay_rate
    final_payout = covered_before_copay - copay_amount
    remaining_limit = limit - covered_before_copay if amount <= limit else 0

    result = {
        "submittedAmount": amount,
        "coveredAmount": covered_before_copay,
        "copayAmount": copay_amount,
        "finalPayout": final_payout,
        "remainingLimit": remaining_limit,
        "exceededLimit": amount > limit
    }
    return json.dumps(result, ensure_ascii=False)

def verifyDocument(documentId: str) -> str:
    """Trả về loại tài liệu, trạng thái hoàn thiện và các lỗi phát sinh."""
    doc = DOCUMENTS_DB.get(documentId)
    if not doc:
        return json.dumps({"error": f"Document ID {documentId} not found."})
    return json.dumps(doc, ensure_ascii=False)

def checkMedicalNecessity(diagnosis: str, procedures: str) -> str:
    """Trả về việc phương pháp điều trị có phù hợp về mặt y tế đối với chẩn
đoán hay không."""
    is_necessary = MEDICAL_RULES_DB.get((diagnosis, procedures), False)
    return json.dumps({
        "diagnosis": diagnosis,
        "procedures": procedures,
        "isMedicalNecessary": is_necessary,
        "note": "Clinically appropriate" if is_necessary else "Treatment does
not match diagnosis or lacks clinical justification."
    }, ensure_ascii=False)

# Danh sách các tool để đăng ký với LLM Client
AVAILABLE_TOOLS = [
    lookupPolicy,
    calculateBenefit,
    verifyDocument,
    checkMedicalNecessity
]
```

3. System Prompt cho AI Agent (system_prompt.py)

System Prompt này giúp ép buộc AI tuân thủ nghiêm ngặt các quy tắc hành vi, thứ tự gọi công cụ và định

dạng báo cáo theo yêu cầu của đề bài.

```
CLAIM_AGENT_SYSTEM_PROMPT = """
Bạn là một Trợ lý AI chuyên đánh giá hồ sơ bồi thường bảo hiểm (Claim
Assessment AI Agent). Nhiệm vụ của bạn là xem xét các yêu cầu bồi thường, gọi
các công cụ hỗ trợ theo đúng quy trình và đưa ra báo cáo đánh giá cấu trúc rõ
ràng.

### QUY TẮC HÀNH VI BẮT BUỘC:
1. **Thứ tự gọi công cụ**: Bạn phải gọi công cụ theo đúng trình tự logic sau
đây[cite: 1]:
   - Bước 1: Gọi `verifyDocument` cho **tất cả** các tài liệu được cung cấp
trong hồ sơ (không được bỏ sót bất kỳ tài liệu nào)[cite: 1].
   - Bước 2: Gọi `lookupPolicy` để kiểm tra thông tin hợp đồng và xác thực thời
hạn hiệu lực[cite: 1].
   - Bước 3: Gọi `checkMedicalNecessity` để kiểm tra tính hợp lý về mặt y tế
giữa chẩn đoán và phương pháp điều trị[cite: 1].
   - Bước 4: Gọi `calculateBenefit` để tính toán số tiền chi trả, đồng chi trả
và hạn mức[cite: 1].

2. **Quy tắc về tài liệu thiếu**:
   - Nếu có bất kỳ tài liệu bắt buộc nào bị thiếu hoặc ở trạng thái
`incomplete`, bạn **PHẢI** đưa ra kiến nghị là `REQUEST_MORE_INFO` (Yêu cầu bổ
sung thông tin), **tuyệt đối không được từ chối (REJECT)** hồ sơ chỉ vì thiếu
giấy tờ[cite: 1].

3. **Ràng buộc về dữ liệu và hạn mức**:
   - Không được tự bịa đặt điều khoản (hallucinate) — mọi thông tin về chính
sách phải lấy từ kết quả của `lookupPolicy`[cite: 1].
   - Không được phê duyệt số tiền vượt quá hạn mức quyền lợi[cite: 1].

### CẤU TRÚC BÁO CÁO ĐÁNH GIÁ (Assessment Report):
Báo cáo đầu ra của bạn phải bao gồm đúng 6 phần sau[cite: 1]:
1. **Document Review**: Liệt kê từng tài liệu, trạng thái của nó
(complete/incomplete/missing) và các lỗi phát hiện[cite: 1].
2. **Policy Verification**: Xác nhận chính sách còn hiệu lực, thành viên được
bảo hiểm và loại hình bồi thường có nằm trong danh mục không[cite: 1].
3. **Medical Necessity**: Đánh giá phương pháp điều trị có phù hợp với chẩn
đoán hay không[cite: 1].
4. **Benefit Calculation**: Số tiền đã nộp, số tiền được bảo hiểm chi trả,
tiền đồng chi trả (copay) và trách nhiệm của thành viên[cite: 1].
5. **Recommendation**: Đưa ra một trong ba kết luận: `APPROVE`, `REJECT`, hoặc
`REQUEST_MORE_INFO` kèm theo lý do cụ thể[cite: 1].
6. **Policy Citations**: Trích dẫn cụ thể điều khoản/mã điều khoản chính sách
cho mọi nhận định trong kiến nghị[cite: 1].
"""
```

User prompt: Vẽ sơ đồ các step hoạt động của Code và AI agent

Response: Dưới đây là sơ đồ mô tả các bước (steps) hoạt động phối hợp giữa Code (Hệ thống/Ứng dụng truyền thống) và AI Agent trong một chu trình xử lý yêu cầu (như bài toán Claim Assessment):



```
+--------------------------------+
```

Chi tiết các bước hoạt động tuần tự:

1. **User Input**: Người dùng hoặc hệ thống bên ngoài gửi một yêu cầu cần xử lý (ví dụ: một hồ sơ claim bảo hiểm cần thẩm định).
2. **Code Initialization (Orchestrator)**: Đoạn code chính khởi tạo môi trường, nạp System Prompt quy định rõ quy tắc (ví dụ: bắt buộc phải gọi tool theo thứ tự, không được bỏ sót tài liệu) và cấu hình các function tools có sẵn.
3. **AI Agent Reasoning**: LLM nhận prompt và dữ liệu đầu vào, phân tích và tự động quyết định xem cần gọi công cụ nào đầu tiên (thường bắt đầu từ verifyDocument).
4. **Code Tool Execution**:
    - Đoạn code kiểm tra yêu cầu gọi tool từ Agent, định tuyến đến hàm Python tương ứng (truy vấn Database giả lập, tính toán toán học, v.v.).
    - Code thực thi logic lập trình thuần túy và trả lại kết quả dạng dữ liệu (JSON) cho Agent.
5. **Agent Loop (Vòng lặp phản hồi)**: Agent tiếp tục đọc kết quả trả về từ tool, đối chiếu với quy tắc để thực hiện bước tiếp theo (ví dụ: sau khi verify xong thì gọi lookupPolicy, rồi checkMedicalNecessity, rồi calculateBenefit).
6. **Final Report Generation**: Sau khi thu thập đủ dữ liệu từ tất cả các tool calls, AI Agent tổng hợp thông tin, đưa ra kết luận (APPROVE, REJECT, hoặc REQUEST_MORE_INFO) kèm theo các trích dẫn điều khoản chính xác.
7. **Response to User**: Hệ thống code nhận báo cáo cuối cùng từ Agent và trả về hiển thị cho người đánh giá hoặc lưu vào hệ thống.

User prompt: Tôi muốn viết code và chạy thử này trên máy tính M2 tôi chưa biết về python, bạn có thể hướng dẫn giúp tôi?

Response: Chào bạn, việc bắt đầu với Python trên máy Mac M2 rất nhanh chóng và thuận lợi. Mặc dù Python có thể mới với bạn, nhưng với nền tảng kỹ thuật sẵn có, bạn sẽ làm quen rất nhanh.

Dưới đây là hướng dẫn từng bước để bạn thiết lập môi trường và chạy thử hệ thống quản lý bồi thường bảo hiểm (Claim Assessment Agent) ngay trên máy Mac M2 của mình.

Bước 1: Kiểm tra và cài đặt Python trên Mac M2

1. Mở ứng dụng **Terminal** trên Mac (bạn có thể nhấn Command + Space, gõ Terminal và nhấn Enter).
2. Kiểm tra xem máy đã có sẵn Python chưa bằng lệnh:
   `python3 --version`
    - Nếu máy hiển thị phiên bản (ví dụ: Python 3.10.x hoặc mới hơn), bạn có thể chuyển sang Bước 2.
    - Nếu máy chưa có hoặc yêu cầu cài đặt công cụ dòng lệnh (Command Line Tools), hãy nhấn cài đặt theo bảng thông báo hiện lên, hoặc cài đặt nhanh qua Homebrew:
      `brew install python`

Bước 2: Tạo thư mục dự án và môi trường ảo (Virtual Environment)

Việc sử dụng môi trường ảo giúp cách ly các thư viện của dự án này với hệ thống chính của Mac.

1. Tạo một thư mục mới cho dự án trên Desktop (hoặc nơi bạn muốn) và di chuyển vào đó:
   ```
   mkdir claim-assessment-agent
   cd claim-assessment-agent
   ```

2. Tạo môi trường ảo có tên là venv:
   ```
   python3 -m venv venv
   ```

3. Kích hoạt môi trường ảo:
   ```
   source venv/bin/activate
   ```

   (Sau khi kích hoạt, bạn sẽ thấy tên (venv) xuất hiện ở đầu dòng lệnh trong Terminal).

Bước 3: Tạo các file mã nguồn

Bạn có thể sử dụng trình soạn thảo code sẵn có trên máy (như **Cursor**, VS Code) hoặc tạo file trực tiếp từ Terminal. Ở đây, mình hướng dẫn tạo nhanh các file cấu trúc như đã thảo luận ở phần trước.

1. **Tạo file cơ sở dữ liệu giả lập (mock_database.py)**: Tạo file và dán đoạn mã cơ sở dữ liệu vào:
   ```
   cat << 'EOF' > mock_database.py
   POLICIES_DB = {
       "POL-001": {
           "policyId": "POL-001",
           "holderName": "Nguyen Van A",
           "status": "ACTIVE",
           "benefits": {
               "Inpatient": {"limit": 50000000, "copay": 0.1},
               "Outpatient": {"limit": 10000000, "copay": 0.2}
           }
       }
   }

   DOCUMENTS_DB = {
       "DOC-101": {
           "documentId": "DOC-101",
           "type": "Medical Bill",
           "status": "complete",
           "issues": []
       }
   }

   MEDICAL_RULES_DB = {
       ("Acute Appendicitis", "Appendectomy"): True
   }
   EOF
   ```

2. **Tạo file chứa 4 công cụ (agent_tools.py)**:
   ```
   cat << 'EOF' > agent_tools.py
   import json
   from mock_database import POLICIES_DB, DOCUMENTS_DB, MEDICAL_RULES_DB

   def lookupPolicy(policyId: str) -> str:
   ```

```
        policy = POLICIES_DB.get(policyId)
        if not policy:
            return json.dumps({"error": f"Policy {policyId} not found."})
        return json.dumps(policy, ensure_ascii=False)

    def calculateBenefit(policyId: str, claimType: str, amount: float) -> str:
        policy = POLICIES_DB.get(policyId)
        if not policy:
            return json.dumps({"error": f"Policy not found."})
        benefit_info = policy["benefits"].get(claimType)
        limit = benefit_info["limit"]
        copay_rate = benefit_info["copay"]

        covered = min(amount, limit)
        copay_amount = covered * copay_rate
        final_payout = covered - copay_amount

        return json.dumps({
            "submittedAmount": amount,
            "coveredAmount": covered,
            "copayAmount": copay_amount,
            "finalPayout": final_payout
        }, ensure_ascii=False)

    def verifyDocument(documentId: str) -> str:
        doc = DOCUMENTS_DB.get(documentId)
        if not doc:
            return json.dumps({"error": f"Document not found."})
        return json.dumps(doc, ensure_ascii=False)

    def checkMedicalNecessity(diagnosis: str, procedures: str) -> str:
        is_necessary = MEDICAL_RULES_DB.get((diagnosis, procedures), False)
        return json.dumps({
            "diagnosis": diagnosis,
            "procedures": procedures,
            "isMedicalNecessary": is_necessary
        }, ensure_ascii=False)
    EOF
```

Bước 4: Tạo file chạy thử (main.py)

Tạo một file kiểm tra nhanh xem các hàm công cụ hoạt động như thế nào:

```
cat << 'EOF' > main.py
from agent_tools import lookupPolicy, calculateBenefit, verifyDocument,
checkMedicalNecessity

if __name__ == "__main__":
    print("=== KIỂM TRA HỆ THỐNG TOOLS (CLAIM ASSESSMENT) ===")
```

```
    # 1. Test verifyDocument
    print("\n[1] Kiểm tra tài liệu DOC-101:")
    print(verifyDocument("DOC-101"))

    # 2. Test lookupPolicy
    print("\n[2] Tra cứu chính sách POL-001:")
    print(lookupPolicy("POL-001"))

    # 3. Test checkMedicalNecessity
    print("\n[3] Kiểm tra tính cần thiết y tế:")
    print(checkMedicalNecessity("Acute Appendicitis", "Appendectomy"))

    # 4. Test calculateBenefit
    print("\n[4] Tính toán quyền lợi (15 triệu VNĐ nội trú):")
    print(calculateBenefit("POL-001", "Inpatient", 15000000))
EOF
```

Bước 5: Chạy chương trình trên Terminal

Tại cửa sổ Terminal (đang ở trong thư mục dự án và đã kích hoạt môi trường ảo), bạn chạy lệnh:

```
python3 main.py
```

Bạn sẽ thấy kết quả trả về ngay lập tức trên màn hình Terminal. Đây là bước nền tảng đầu tiên để kiểm chứng logic Python trước khi tích hợp vào vòng lặp gọi AI Agent hoàn chỉnh.

User prompt: Bước tiếp theo là làm gì?

Response: Sau khi bạn đã thiết lập thành công cấu trúc cơ sở dữ liệu và các hàm công cụ (tools) cơ bản trên máy Mac M2, bước tiếp theo chúng ta sẽ tiến hành **tích hợp mô hình ngôn ngữ lớn (LLM)** và **xây dựng vòng lặp điều khiển Agent (Agent Loop)** để hệ thống có thể tự động gọi các công cụ này theo đúng ngữ cảnh.

Dưới đây là các bước tiếp theo cụ thể bạn cần thực hiện:

Bước 1: Cài đặt thư viện LLM SDK và cấu hình API Key

Tùy thuộc vào việc bạn muốn sử dụng nhà cung cấp nào (Anthropic Claude, OpenAI, hoặc Google Gemini), bạn cần cài đặt thư viện tương ứng. Ví dụ, nếu bạn dùng **Anthropic Claude** hoặc **OpenAI**:

1. Cài đặt thư viện trong môi trường ảo (venv):
   `pip install anthropic openai python-dotenv`

2. Tạo file .env để lưu trữ khóa API của bạn một cách bảo mật:
   `echo "ANTHROPIC_API_KEY=your_actual_api_key_here" > .env`

Bước 2: Xây dựng 3 Test Cases mẫu (Theo yêu cầu đề bài)

Theo yêu cầu của thử thách, bạn cần tạo 3 bộ dữ liệu test hoàn chỉnh bao gồm chi tiết yêu cầu bồi thường, dữ liệu chính sách, danh sách tài liệu và kết quả mong đợi:

- **Test Case 1 (Phê duyệt trực tiếp):** Hồ sơ hợp lệ, đủ giấy tờ → Kỳ vọng kết quả: APPROVE.
- **Test Case 2 (Từ chối):** Hồ sơ vi phạm điều khoản loại trừ hoặc vượt hạn mức → Kỳ vọng kết quả: REJECT.
- **Test Case 3 (Yêu cầu bổ sung):** Hồ sơ thiếu tài liệu bắt buộc → Kỳ vọng kết quả: REQUEST_MORE_INFO (không được từ chối).

Bước 3: Viết mã nguồn cho Agent Runner (agent_runner.py)

Tạo một file điều phối chính để:

1. Gửi System Prompt và yêu cầu bồi thường của người dùng tới LLM.
2. Xử lý phản hồi từ LLM khi mô hình muốn gọi các tool (verifyDocument, lookupPolicy, v.v.).
3. Thực thi hàm Python tương ứng và trả kết quả ngược lại cho LLM.
4. Tổng hợp thành **Báo cáo đánh giá (Assessment Report)** gồm 6 phần chuẩn (Document Review, Policy Verification, Medical Necessity, Benefit Calculation, Recommendation, Policy Citations).

Bạn có muốn tôi viết mã nguồn mẫu cho phần **Agent Runner** kết nối LLM và xử lý vòng lặp gọi tool này ngay bây giờ không?

User prompt: Vâng

Response: Dưới đây là hướng dẫn chi tiết từng bước để bạn hoàn thiện phần kết nối LLM API và vòng lặp xử lý công cụ (Agent Runner) trên máy Mac M2 của bạn.

Bước 1: Chuẩn bị môi trường & API Key

Mở **Terminal**, đảm bảo bạn đang ở trong thư mục claim-assessment-agent và đã kích hoạt môi trường ảo (venv).

1. **Cài đặt thư viện openai và python-dotenv**:
   ```
   pip install openai python-dotenv
   ```

   (Ghi chú: Thư viện openai có thể dùng cho OpenAI API, hoặc dùng kết nối với Gemini Open-AI compatible, DeepSeek, Claude qua router/OpenAI format, giúp code rất gọn).
2. **Tạo file .env chứa API Key**: Thay YOUR_API_KEY_HERE bằng API Key thực tế của bạn:
   ```
   echo "OPENAI_API_KEY=YOUR_API_KEY_HERE" > .env
   ```

Bước 2: Chuẩn bị 3 Test Cases mẫu (test_cases.py)

Tạo file test_cases.py để chứa 3 kịch bản kiểm thử theo đúng yêu cầu đề bài:

```
cat << 'EOF' > test_cases.py
TEST_CASES = [
    {
        "id": "CASE-1",
        "name": "Approval - Hồ sơ hợp lệ",
        "prompt": "Hãy thẩm định hồ sơ bồi thường cho bệnh nhân Nguyen Van A.
Mã hợp đồng: POL-001. Loại hình: Inpatient. Số tiền yêu cầu: 15,000,000 VNĐ.
Chẩn đoán: Acute Appendicitis. Phương pháp điều trị: Appendectomy. Các tài liệu
đã nộp: DOC-101 (Medical Bill)."
    },
    {
        "id": "CASE-2",
        "name": "Rejection - Vượt hạn mức hoặc loại trừ",
        "prompt": "Hãy thẩm định hồ sơ bồi thường cho bệnh nhân Nguyen Van A.
Mã hợp đồng: POL-001. Loại hình: Inpatient. Số tiền yêu cầu: 80,000,000 VNĐ
(vượt hạn mức 50tr). Chẩn đoán: Acute Appendicitis. Phương pháp điều trị:
Appendectomy. Các tài liệu đã nộp: DOC-101 (Medical Bill)."
    },
    {
        "id": "CASE-3",
        "name": "Request More Info - Thiếu / incomplete tài liệu",
        "prompt": "Hãy thẩm định hồ sơ bồi thường cho bệnh nhân Nguyen Van A.
Mã hợp đồng: POL-001. Loại hình: Inpatient. Số tiền yêu cầu: 10,000,000 VNĐ.
Chẩn đoán: Acute Appendicitis. Phương pháp điều trị: Appendectomy. Các tài liệu
đã nộp: DOC-102 (Discharge Summary bị thiếu chữ ký/dấu)."
    }
]
EOF
```

Bước 3: Đăng ký schemas cho các công cụ (tool_definitions.py)

Để LLM biết hệ thống có các tool nào và nhận tham số gì, chúng ta cần định nghĩa schema dạng JSON:

```
cat << 'EOF' > tool_definitions.py
TOOLS_SCHEMA = [
    {
        "type": "function",
        "function": {
            "name": "verifyDocument",
            "description": "Kiểm tra loại tài liệu, trạng thái hoàn thiện và
các lỗi phát sinh.",
            "parameters": {
                "type": "object",
                "properties": {
                    "documentId": {"type": "string", "description": "Mã tài
liệu cần kiểm tra, ví dụ: DOC-101"}
                },
                "required": ["documentId"]
```

```
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "lookupPolicy",
            "description": "Tra cứu quyền lợi, hạn mức, điều khoản loại trừ và
đồng chi trả của hợp đồng bảo hiểm.",
            "parameters": {
                "type": "object",
                "properties": {
                    "policyId": {"type": "string", "description": "Mã hợp đồng
bảo hiểm, ví dụ: POL-001"}
                },
                "required": ["policyId"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "checkMedicalNecessity",
            "description": "Kiểm tra sự phù hợp về mặt y tế giữa chẩn đoán
bệnh và phương pháp điều trị.",
            "parameters": {
                "type": "object",
                "properties": {
                    "diagnosis": {"type": "string", "description": "Chẩn đoán
bệnh"},
                    "procedures": {"type": "string", "description": "Phương
pháp/Phác đồ điều trị"}
                },
                "required": ["diagnosis", "procedures"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "calculateBenefit",
            "description": "Tính toán số tiền được chi trả, số tiền đồng chi
trả (copay) và chi phí người tham gia bảo hiểm phải trả.",
            "parameters": {
                "type": "object",
                "properties": {
                    "policyId": {"type": "string", "description": "Mã hợp đồng
bảo hiểm"},
                    "claimType": {"type": "string", "description": "Loại bồi
thường: Inpatient hoặc Outpatient"},
                    "amount": {"type": "number", "description": "Số tiền yêu
cầu bồi thường"}
```

```
                },
                "required": ["policyId", "claimType", "amount"]
            }
        }
    }
]
EOF
```

Bước 4: Viết Agent Runner (agent_runner.py)

Đây là trái tim của hệ thống: điều khiển vòng lặp Function Calling, ép LLM đi theo đúng thứ tự logic và in ra log từng bước.

```
cat << 'EOF' > agent_runner.py
import os
import json
from dotenv import load_dotenv
from openai import OpenAI
import agent_tools
from tool_definitions import TOOLS_SCHEMA

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# Mapping tên function với hàm Python thực tế
TOOL_MAPPING = {
    "verifyDocument": agent_tools.verifyDocument,
    "lookupPolicy": agent_tools.lookupPolicy,
    "checkMedicalNecessity": agent_tools.checkMedicalNecessity,
    "calculateBenefit": agent_tools.calculateBenefit
}

SYSTEM_PROMPT = """
Bạn là một AI Agent thẩm định hồ sơ bồi thường bảo hiểm chuyên nghiệp.
Nhiệm vụ của bạn là thực hiện đánh giá hồ sơ theo đúng thứ tự logic bắt buộc:
1. Xác minh tất cả các tài liệu (verifyDocument)
2. Tra cứu điều khoản bảo hiểm (lookupPolicy)
3. Kiểm tra tính cần thiết y tế (checkMedicalNecessity)
4. Tính toán mức chi trả quyền lợi (calculateBenefit)

QUY TẮC QUAN TRỌNG:
- Nếu có bất kỳ tài liệu nào bị thiếu hoặc chưa hoàn thiện (incomplete), bạn
PHẢI đưa ra kiến nghị là REQUEST_MORE_INFO (không được REJECT chỉ vì thiếu
giấy tờ).
- Không tự bịa đặt điều khoản bảo hiểm.
- Mọi nhận định phải dẫn chiếu điều khoản cụ thể.

ĐẦU RA BÁO CÁO (ASSESSMENT REPORT) BAO GỒM 6 PHẦN:
1. Document Review
2. Policy Verification
```

```
3. Medical Necessity
4. Benefit Calculation
5. Recommendation (APPROVE / REJECT / REQUEST_MORE_INFO)
6. Policy Citations
"""

def run_claim_agent(user_prompt: str):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_prompt}
    ]

    print("\n" + "="*60)
    print("🚀 BẮT ĐẦU QUÁ TRÌNH THẨM ĐỊNH CLAIM")
    print("="*60)

    while True:
        # Gọi LLM cùng với danh sách Tools
        response = client.chat.completions.create(
            model="gpt-4o",  # hoặc gpt-4o-mini
            messages=messages,
            tools=TOOLS_SCHEMA,
            tool_choice="auto"
        )

        response_message = response.choices[0].message
        messages.append(response_message)

        # Nếu LLM muốn gọi Tool (Tool Calls)
        if response_message.tool_calls:
            for tool_call in response_message.tool_calls:
                fn_name = tool_call.function.name
                fn_args = json.loads(tool_call.function.arguments)

                print(f"\n🛠️  [TOOL CALL]: {fn_name}")
                print(f"📥 Input: {fn_args}")

                # Thực thi hàm Python tương ứng
                tool_func = TOOL_MAPPING.get(fn_name)
                if tool_func:
                    result = tool_func(**fn_args)
                else:
                    result = json.dumps({"error": "Unknown tool"})

                print(f"📤 Output: {result}")

                # Phản hồi lại kết quả của Tool cho LLM
                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result
                })
```

```
        else:
            # LLM đã hoàn tất gọi tool và đưa ra báo cáo cuối cùng
            print("\n" + "="*60)
            print("📋 BÁO CÁO ĐÁNH GIÁ CUỐI CÙNG (ASSESSMENT REPORT)")
            print("="*60)
            print(response_message.content)
            break

if __name__ == "__main__":
    from test_cases import TEST_CASES
    # Chạy thử Case 1
    run_claim_agent(TEST_CASES[0]["prompt"])
EOF
```

Bước 5: Chạy thử trên máy Mac M2

Chạy lệnh sau tại Terminal:

```
python3 agent_runner.py
```

Kết quả thu được: Bạn sẽ thấy trên Terminal in ra toàn bộ lịch sử suy luận (Reasoning Trace) của Agent: từ việc tự động gọi verifyDocument → lookupPolicy → checkMedicalNecessity → calculateBenefit, và cuối cùng in ra báo cáo hoàn chỉnh có cấu trúc 6 phần với trích dẫn điều khoản chính xác.

Bạn có gặp khó khăn hay lỗi nào trong quá trình cài đặt và chạy thử không?

User prompt: Nếu tôi không có OPENAI_API_KEY thì phải làm sao?

Response: Nếu bạn không có OPENAI_API_KEY, bạn hoàn toàn có thể sử dụng các giải pháp thay thế **hoàn toàn miễn phí** và rất phù hợp với phần cứng của bạn:

Cách 1: Chạy mô hình cục bộ bằng Ollama (Miễn phí 100%, không cần API Key)

Vì bạn đang sử dụng **MacBook Pro M2 với 32GB unified memory**, máy của bạn hoàn toàn đủ mạnh để chạy mượt mà các mô hình mã hóa/lập trình cục bộ thông qua **Ollama**. Cách này giúp bạn không tốn chi phí, bảo mật tuyệt đối và không cần bất kỳ API key nào từ các hãng lớn.

1. Đảm bảo Ollama đang chạy trên máy Mac của bạn. Bạn có thể tải một model chuyên code (ví dụ qwen2.5-coder hoặc deepseek-coder) bằng lệnh Terminal:
   `ollama run qwen2.5-coder:7b`

2. Trong file agent_runner.py, bạn chỉ cần cấu hình lại đối tượng OpenAI client để trỏ vào endpoint local của Ollama thay vì OpenAI cloud:
   ```
   from openai import OpenAI

   # Kết nối trực tiếp với Ollama chạy local (không cần key thật)
   ```

```
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"  # Ollama không yêu cầu key thực
)
```

3. Khi gọi hàm tạo completion trong agent_runner.py, bạn đổi tên model thành tên model bạn đã tải về trên Ollama:

```
response = client.chat.completions.create(
    model="qwen2.5-coder:7b",  # Tên model bạn có trong Ollama
    messages=messages,
    tools=TOOLS_SCHEMA,
    tool_choice="auto"
)
```

Cách 2: Sử dụng Google Gemini API (Có gói Free-tier miễn phí)

Google cung cấp **Google AI Studio** nơi bạn có thể đăng ký một **Gemini API Key hoàn toàn miễn phí** với hạn mức (rate limit) rất thoải mái cho việc học tập, làm challenge và thử nghiệm.

1. Truy cập trang Google AI Studio và đăng nhập bằng tài khoản Google của bạn để lấy một API Key miễn phí.
2. Lưu key đó vào file .env dưới tên biến tùy ý (hoặc GEMINI_API_KEY).
3. Google Gemini API cũng hỗ trợ thư viện openai hoặc bạn có thể dùng thư viện chính thức google-genai để tích hợp.

Bạn muốn thử cấu hình chạy local với **Ollama** trên chiếc M2 Pro của mình hay muốn hướng dẫn lấy **Gemini API Key miễn phí**?

User prompt: tôi chạy bị lỗi là do sao?

Response: Lỗi này xảy ra vì bạn đã **sao chép nhầm câu lệnh của Terminal** (dòng lệnh tạo file như cat << 'EOF' > main.py) trực tiếp **vào bên trong file Python (main.py)**.

Khi bạn chạy lệnh python3 main.py, Python đọc file này và gặp từ cat, nhưng đây là một lệnh của hệ điều hành Unix chứ không phải từ khóa hay biến trong Python, dẫn đến lỗi:

NameError: name 'cat' is not defined (Python không hiểu cat là gì).

Cách khắc phục:

Bước 1: Mở và sửa lại file main.py

Bạn hãy mở file main.py bằng một trình soạn thảo code (như VS Code, Cursor) hoặc xóa sạch nội dung cũ của file đó đi, sau đó chỉ dán **mã nguồn Python thuần túy** vào.

Ví dụ, nội dung chuẩn bên trong file main.py chỉ nên gồm các dòng code Python như thế này (không có chữ

cat << 'EOF' ở đầu):

```
from agent_tools import lookupPolicy, calculateBenefit, verifyDocument,
checkMedicalNecessity

if __name__ == "__main__":
    print("=== KIỂM TRA HỆ THỐNG TOOLS (CLAIM ASSESSMENT) ===")

    # 1. Test verifyDocument
    print("\n[1] Kiểm tra tài liệu DOC-101:")
    print(verifyDocument("DOC-101"))

    # 2. Test lookupPolicy
    print("\n[2] Tra cứu chính sách POL-001:")
    print(lookupPolicy("POL-001"))
```

Bước 2: Chạy lại trên Terminal

Sau khi đã sửa lại file main.py cho đúng định dạng Python, bạn quay lại cửa sổ Terminal và chạy lại lệnh:

```
python3 main.py
```

Lưu ý nhỏ: Các câu lệnh bắt đầu bằng cat << 'EOF' > tên_file là lệnh dùng để **gõ trực tiếp vào Terminal** để tạo file nhanh, chứ không phải là nội dung bên trong file Python. Bạn chỉ dán những đoạn đó vào cửa sổ Terminal đang mở, chứ không dán vào bên trong các file .py nhé!

User prompt: lỗi này là vì sao?

Response: Lỗi này xảy ra vì môi trường ảo (venv) của bạn **chưa cài đặt thư viện python-dotenv** (thư viện dùng để đọc file .env).

Python báo lỗi: ModuleNotFoundError: No module named 'dotenv' có nghĩa là máy không tìm thấy thư viện này để thực thi câu lệnh from dotenv import load_dotenv.

Cách khắc phục:

Bạn chỉ cần chạy lệnh cài đặt thư viện này trực tiếp trên cửa sổ Terminal (đảm bảo môi trường ảo (venv) đang được kích hoạt):

```
pip install python-dotenv openai
```

Sau khi cài đặt xong, bạn chạy lại file của mình:

```
python3 agent_runner.py
```

Lệnh này sẽ tải và cài đặt gói thư viện cần thiết vào môi trường hiện tại của bạn, và lỗi sẽ được giải quyết triệt để!

User prompt: tôi chạy rồi và kết quả ra như hình thì bị sao?

Response: Nhìn vào hình ảnh kết quả của bạn, chương trình **không hề bị lỗi (không có lỗi traceback đỏ)**, mà đây là một **hiện tượng logic trong vòng lặp của AI Agent**.

Nguyên nhân xảy ra hiện tượng này:

1. Model AI (khi bạn gọi mô hình) đã nhận được prompt và quyết định gọi công cụ đầu tiên: {"name": "verifyDocument", "arguments": {"documentId": "DOC-101"}}.
2. Tuy nhiên, trong đoạn code agent_runner.py mẫu trước đó, đoạn code chỉ thực thi lệnh gọi tool **một lần ở lượt đầu tiên** mà chưa tạo một vòng lặp liên tục (while loop đúng chuẩn) để **nhận kết quả trả về từ tool, gửi ngược lại cho AI**, và đợi AI tiếp tục gọi các tool tiếp theo (lookupPolicy, checkMedicalNecessity,...).
3. Do đó, AI vừa mới yêu cầu gọi tool verifyDocument, code thực hiện xong nhưng không gửi lại kết quả cho AI, khiến chương trình dừng ngay tại đó và in ra câu lệnh gọi tool cuối cùng rồi kết thúc.

Cách khắc phục triệt để:

Bạn hãy mở file agent_runner.py và cập nhật lại đoạn hàm run_claim_agent để nó tự động gửi kết quả tool trở lại cho AI (vòng lặp đệ quy/while hoàn chỉnh).

Dưới đây là đoạn code chuẩn hoàn chỉnh cho agent_runner.py:

```
import os
import json
from dotenv import load_dotenv
from openai import OpenAI
import agent_tools
from tool_definitions import TOOLS_SCHEMA

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

TOOL_MAPPING = {
    "verifyDocument": agent_tools.verifyDocument,
    "lookupPolicy": agent_tools.lookupPolicy,
    "checkMedicalNecessity": agent_tools.checkMedicalNecessity,
    "calculateBenefit": agent_tools.calculateBenefit
}

SYSTEM_PROMPT = """
Bạn là một AI Agent thẩm định hồ sơ bồi thường bảo hiểm chuyên nghiệp.
Nhiệm vụ của bạn là thực hiện đánh giá hồ sơ theo đúng thứ tự logic bắt buộc:
```

```
1. Xác minh tất cả các tài liệu (verifyDocument)
2. Tra cứu điều khoản bảo hiểm (lookupPolicy)
3. Kiểm tra tính cần thiết y tế (checkMedicalNecessity)
4. Tính toán mức chi trả quyền lợi (calculateBenefit)

QUY TẮC QUAN TRỌNG:
- Nếu có bất kỳ tài liệu nào bị thiếu hoặc chưa hoàn thiện (incomplete), bạn
PHẢI đưa ra kiến nghị là REQUEST_MORE_INFO.
- Không tự bịa đặt điều khoản bảo hiểm.
- Đầu ra phải bao gồm 6 phần: Document Review, Policy Verification, Medical
Necessity, Benefit Calculation, Recommendation, Policy Citations.
"""

def run_claim_agent(user_prompt: str):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_prompt}
    ]

    print("\n" + "="*60)
    print("🚀 BẮT ĐẦU QUÁ TRÌNH THẨM ĐỊNH CLAIM")
    print("="*60)

    # Vòng lặp cho phép AI gọi nhiều tool liên tiếp cho đến khi có kết quả
cuối cùng
    while True:
        response = client.chat.completions.create(
            model="gpt-4o",  # Hoặc tên model bạn đang dùng
            messages=messages,
            tools=TOOLS_SCHEMA,
            tool_choice="auto"
        )

        response_message = response.choices[0].message
        messages.append(response_message)

        # Kiểm tra xem AI có muốn gọi Tool tiếp hay không
        if response_message.tool_calls:
            for tool_call in response_message.tool_calls:
                fn_name = tool_call.function.name
                fn_args = json.loads(tool_call.function.arguments)

                print(f"\n🛠️ [TOOL CALL]: {fn_name} | Arguments: {fn_args}")

                tool_func = TOOL_MAPPING.get(fn_name)
                if tool_func:
                    result = tool_func(**fn_args)
                else:
                    result = json.dumps({"error": "Unknown tool"})

                print(f"📤 [TOOL OUTPUT]: {result}")
```

```
                # QUAN TRỌNG: Gửi kết quả tool ngược lại vào mảng messages để
AI đọc tiếp
                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result
                })
        else:
            # Khi AI đã gọi xong tất cả các tool và đưa ra báo cáo cuối cùng
            print("\n" + "="*60)
            print("📋 BÁO CÁO ĐÁNH GIÁ CUỐI CÙNG (ASSESSMENT REPORT)")
            print("="*60)
            print(response_message.content)
            break

if __name__ == "__main__":
    from test_cases import TEST_CASES
    run_claim_agent(TEST_CASES[0]["prompt"])
```

Sau khi cập nhật lại file agent_runner.py với vòng lặp gửi nhận tool đầy đủ này, bạn chạy lại lệnh python3 agent_runner.py, AI sẽ tự động gọi lần lượt cả 4 công cụ và in ra toàn bộ báo cáo hoàn chỉnh!

User prompt: tôi đang chạy với ollama & qwen2.5 coder 7b

Response: Tuyệt vời! Việc bạn đang dùng **Ollama** kết hợp với **Qwen2.5 Coder 7B** trên chiếc MacBook Pro M2 (32GB) là một lựa chọn cực kỳ thông minh, miễn phí 100%, chạy hoàn toàn cục bộ và bảo mật tuyệt đối.

Tuy nhiên, có một điểm rất quan trọng cần lưu ý khi dùng các mô hình chạy local như Qwen2.5-Coder (hoặc các model open-weight nhỏ dưới 30B) với chức năng **Function Calling (Tools)**:

Nhiều model local đôi khi không tự động kích hoạt vòng lặp gọi tool (tool calling loop) mượt mà như OpenAI GPT-4o hoặc Claude.

Nếu bạn chạy file agent_runner.py với Ollama mà bị dừng ở dòng [TOOL CALL] hoặc model chỉ in ra text thuần mà không gọi tool, chúng ta cần cấu hình lại một chút để đảm bảo model hiểu đúng cách gọi hàm.

Cách cấu hình agent_runner.py để chạy chuẩn xác với Ollama & Qwen2.5-Coder:

Bạn hãy mở file agent_runner.py và điều chỉnh lại phần khởi tạo client để trỏ vào Ollama local như sau:

```
import os
import json
from openai import OpenAI
import agent_tools
from tool_definitions import TOOLS_SCHEMA
```

```
# Khởi tạo OpenAI Client trỏ trực tiếp vào Ollama chạy trên máy Mac của bạn
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"  # Ollama không yêu cầu API key thực
)

TOOL_MAPPING = {
    "verifyDocument": agent_tools.verifyDocument,
    "lookupPolicy": agent_tools.lookupPolicy,
    "checkMedicalNecessity": agent_tools.checkMedicalNecessity,
    "calculateBenefit": agent_tools.calculateBenefit
}

SYSTEM_PROMPT = """
Bạn là một AI Agent thẩm định hồ sơ bồi thường bảo hiểm.
Nhiệm vụ của bạn là phải gọi các công cụ theo đúng trình tự: verifyDocument ->
lookupPolicy -> checkMedicalNecessity -> calculateBenefit.
Sau khi có đủ kết quả từ các công cụ, hãy tổng hợp thành Báo cáo đánh giá gồm 6
phần: Document Review, Policy Verification, Medical Necessity, Benefit
Calculation, Recommendation, Policy Citations.
"""

def run_claim_agent(user_prompt: str):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_prompt}
    ]

    print("\n" + "="*60)
    print("🚀 BẮT ĐẦU QUÁ TRÌNH THẨM ĐỊNH CLAIM (OLLAMA LOCAL)")
    print("="*60)

    while True:
        try:
            # Gọi model qwen2.5-coder:7b trên Ollama
            response = client.chat.completions.create(
                model="qwen2.5-coder:7b",  # Thay tên model bạn đã pull về
Ollama
                messages=messages,
                tools=TOOLS_SCHEMA,
                tool_choice="auto"
            )
        except Exception as e:
            print(f"❌ Lỗi kết nối Ollama: {e}")
            print("👉 Hãy đảm bảo bạn đã bật ứng dụng Ollama hoặc chạy lệnh
'ollama serve' trong Terminal.")
            break

        response_message = response.choices[0].message
        messages.append(response_message)

        # Kiểm tra xem mô hình có yêu cầu gọi tool không
```

```
        if response_message.tool_calls:
            for tool_call in response_message.tool_calls:
                fn_name = tool_call.function.name
                fn_args = json.loads(tool_call.function.arguments)

                print(f"\n🛠️ [TOOL CALL]: {fn_name} | Arguments: {fn_args}")

                tool_func = TOOL_MAPPING.get(fn_name)
                if tool_func:
                    result = tool_func(**fn_args)
                else:
                    result = json.dumps({"error": "Unknown tool"})

                print(f"📤 [TOOL OUTPUT]: {result}")

                # Đưa kết quả trả về vào message lịch sử để Qwen đọc tiếp
                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result
                })
        else:
            # Khi mô hình đã xử lý xong và đưa ra kết quả cuối cùng
            print("\n" + "="*60)
            print("📋 BÁO CÁO ĐÁNH GIÁ CUỐI CÙNG (ASSESSMENT REPORT)")
            print("="*60)
            print(response_message.content)
            break

if __name__ == "__main__":
    from test_cases import TEST_CASES
    run_claim_agent(TEST_CASES[0]["prompt"])
```

Một số lưu ý khi chạy với Ollama:

1. **Đảm bảo Ollama đang chạy ngầm**: Trước khi chạy lệnh Python, hãy chắc chắn ứng dụng Ollama đã được mở trên Mac, hoặc bạn có thể gõ ollama serve ở một cửa sổ Terminal khác.
2. **Kiểm tra tên model**: Chạy lệnh ollama list trong Terminal để xem chính xác tên model bạn đã tải (ví dụ: qwen2.5-coder:7b hoặc qwen2.5-coder:latest) và điền đúng tên đó vào biến model="..." trong code.

Bạn hãy thử cập nhật lại file agent_runner.py theo mẫu trên và chạy lại lệnh `python3 agent_runner.py xem Qwen2.5-Coder phản hồi thế nào nhé!

User prompt: tôi chạy ra kết quả này là sao?

Response: Nhìn vào hình ảnh kết quả của bạn, **đây là một tín hiệu cực kỳ tốt!** Model Qwen2.5-Coder của

bạn đã thực hiện được một việc rất quan trọng:

Nó đã gọi **đầy đủ cả 4 công cụ (Tool Calls)** theo đúng tuần tự logic mà bạn yêu cầu:

1. verifyDocument (Kiểm tra tài liệu DOC-101)
2. lookupPolicy (Tra cứu chính sách POL-001)
3. checkMedicalNecessity (Kiểm tra y tế Acute Appendicitis & Appendectomy)
4. calculateBenefit (Tính toán số tiền 15,000,000 VNĐ)

Tại sao chương trình lại dừng ở đây mà chưa hiện báo cáo?

Lý do là vì đoạn code trong agent_runner.py sau khi thực hiện xong lần gọi tool cuối cùng, vòng lặp vừa thực thi xong tool calculateBenefit và gửi kết quả về cho Qwen, nhưng chương trình lại kết thúc ngay trước khi kịp in ra câu trả lời tổng kết cuối cùng của mô hình (hoặc Qwen trả về kết quả dạng thô mà đoạn code chưa gom lại để in ra phần nội dung text thuần).

Cách khắc phục để hiển thị đầy đủ Báo cáo đánh giá (Assessment Report):

Bạn chỉ cần tinh chỉnh lại một chút ở đoạn kết thúc của file agent_runner.py để đảm bảo sau khi gọi xong các tool, code sẽ lấy câu trả lời cuối cùng của Qwen và in ra màn hình.

Bạn hãy mở file agent_runner.py và kiểm tra xem đoạn code xử lý khi không còn tool_calls đã in ra response_message.content hay chưa. Dưới đây là đoạn code chuẩn để in ra trọn vẹn kết quả:

```
        # Kiểm tra xem mô hình có muốn gọi tool tiếp không
        if response_message.tool_calls:
            for tool_call in response_message.tool_calls:
                fn_name = tool_call.function.name
                fn_args = json.loads(tool_call.function.arguments)

                print(f"\n🛠️ [TOOL CALL]: {fn_name} | Arguments: {fn_args}")

                tool_func = TOOL_MAPPING.get(fn_name)
                if tool_func:
                    result = tool_func(**fn_args)
                else:
                    result = json.dumps({"error": "Unknown tool"})

                print(f"📤 [TOOL OUTPUT]: {result}")

                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "content": result
                })
        else:
            # ---> ĐOẠN NÀY SẼ IN RA BÁO CÁO KHI QWEN HOÀN TẤT SUY LUẬN <---
            print("\n" + "="*60)
            print("📋 BÁO CÁO ĐÁNH GIÁ CUỐI CÙNG (ASSESSMENT REPORT)")
            print("="*60)
```

```
            if response_message.content:
                print(response_message.content)
            else:
                print("(Mô hình đã hoàn thành gọi tool nhưng chưa tự sinh text
báo cáo. Hãy thêm prompt yêu cầu tổng hợp báo cáo).")
            break
```

Như vậy, mô hình Qwen chạy local trên M2 của bạn đã **hoàn thành xuất sắc phần cốt lõi nhất của Agent** là hiểu và điều hướng gọi đúng các API/Tools. Bạn chỉ cần tinh chỉnh nhỏ phần hiển thị văn bản kết quả bên dưới là sẽ nhận được trọn vẹn Báo cáo 6 phần theo yêu cầu của đề bài!

User prompt: này viết bằng Java có được không hay phải dùng python

Response: Hoàn toàn **được**, bạn không bắt buộc phải dùng Python! Đề bài của AI Challenge 11 chỉ yêu cầu xây dựng logic cho AI Agent và các công cụ, hoàn toàn không giới hạn ngôn ngữ lập trình.

Nếu bạn có thế mạnh về hệ sinh thái **Java**, bạn hoàn toàn có thể sử dụng Java để hoàn thành thử thách này. Các công nghệ và thư viện hiện đại trong Java hỗ trợ việc xây dựng AI Agent và Function Calling rất mạnh mẽ:

- **LangChain4j**: Thư viện tương đương với LangChain của Python, hỗ trợ cực tốt việc kết nối với LLM (cả OpenAI lẫn chạy local qua Ollama), quản lý bộ nhớ và tích hợp Tool Calling thông qua các Interface hoặc Annotation (@Tool).
- **Spring AI**: Dự án chính thức của Spring giúp tích hợp AI vào các ứng dụng Spring Boot một cách chuẩn hóa.
- **Java HTTP Client (Thuần)**: Gửi các HTTP Request trực tiếp đến endpoint của Ollama (http://localhost:11434/v1/chat/completions) bằng mã Java thuần mà không cần cài đặt quá nhiều thư viện phức tạp.

Cách tiếp cận cơ bản khi viết bằng Java:

1. **Viết các hàm công cụ (Tools)**: Xây dựng 4 hàm Java tương ứng (verifyDocument, lookupPolicy, checkMedicalNecessity, calculateBenefit).
2. **Khai báo Tool Schema**: Cung cấp định nghĩa JSON cho các hàm để LLM nhận diện được tham số đầu vào.
3. **Vòng lặp điều khiển (Agent Loop)**: Sử dụng vòng lặp while trong Java để nhận phản hồi từ LLM, kiểm tra xem LLM có gọi tool hay không, thực thi hàm Java tương ứng và gửi kết quả ngược lại cho mô hình cho đến khi nhận được báo cáo hoàn chỉnh.

Bạn có muốn tôi phác thảo một đoạn code mẫu cấu trúc bằng **Java thuần** hoặc sử dụng **LangChain4j** để chạy với Ollama trên máy của bạn không?

User prompt: Viết code bằng Java thuần giúp tôi

Response: Hoàn toàn được! Việc xây dựng AI Agent và tích hợp tool calling bằng **Java thuần (Core Java)** là

một cách tiếp cận cực kỳ mạnh mẽ, tận dụng các tính năng hiện đại có sẵn từ **Java 11+** (java.net.http.HttpClient) để giao tiếp trực tiếp với Ollama (chạy local trên máy Mac M2 của bạn) mà không cần cài đặt quá nhiều thư viện phức tạp.

Dưới đây là mã nguồn mẫu gồm 3 file Java cấu trúc rõ ràng để bạn chạy thử hệ thống **Claim Assessment Agent** bằng Java thuần:

1. Cơ sở dữ liệu giả lập (MockDatabase.java)

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;

public class MockDatabase {

    // Dữ liệu chính sách bảo hiểm
    public static Map<String, Object> getPolicy(String policyId) {
        if ("POL-001".equals(policyId)) {
            Map<String, Object> policy = new HashMap<>();
            policy.put("policyId", "POL-001");
            policy.put("holderName", "Nguyen Van A");
            policy.put("status", "ACTIVE");

            Map<String, Map<String, Object>> benefits = new HashMap<>();
            benefits.put("Inpatient", Map.of("limit", 50000000.0, "copay",
0.1));
            benefits.put("Outpatient", Map.of("limit", 10000000.0, "copay",
0.2));
            policy.put("benefits", benefits);

            return policy;
        }
        return null;
    }

    // Dữ liệu tài liệu
    public static Map<String, Object> getDocument(String documentId) {
        if ("DOC-101".equals(documentId)) {
            return Map.of(
                "documentId", "DOC-101",
                "type", "Medical Bill",
                "status", "complete",
                "issues", List.of()
            );
        } else if ("DOC-102".equals(documentId)) {
            return Map.of(
                "documentId", "DOC-102",
                "type", "Discharge Summary",
                "status", "incomplete",
                "issues", List.of("Missing physician signature")
            );
```

```
        }
        return Map.of("error", "Document not found");
    }

    // Kiểm tra y tế
    public static boolean checkMedicalRule(String diagnosis, String procedures)
{
        return "Acute Appendicitis".equals(diagnosis) &&
"Appendectomy".equals(procedures);
    }
}
```

2. Định nghĩa 4 công cụ (AgentTools.java)

```
public class AgentTools {

    public static String lookupPolicy(String policyId) {
        var policy = MockDatabase.getPolicy(policyId);
        if (policy == null) return "{\"error\": \"Policy not found\"}";
        return policy.toString();
    }

    public static String calculateBenefit(String policyId, String claimType,
double amount) {
        var policy = MockDatabase.getPolicy(policyId);
        if (policy == null) return "{\"error\": \"Policy not found\"}";

        @SuppressWarnings("unchecked")
        var benefits = (Map<String, Map<String, Object>>)
policy.get("benefits");
        var benefitInfo = benefits.get(claimType);
        if (benefitInfo == null) return "{\"error\": \"Claim type not
covered\"}";

        double limit = (double) benefitInfo.get("limit");
        double copayRate = (double) benefitInfo.get("copay");

        double covered = Math.min(amount, limit);
        double copayAmount = covered * copayRate;
        double finalPayout = covered - copayAmount;

        return String.format("{\"submittedAmount\": %.1f, \"coveredAmount\":
%.1f, \"copayAmount\": %.1f, \"finalPayout\": %.1f}",
                amount, covered, copayAmount, finalPayout);
    }

    public static String verifyDocument(String documentId) {
        var doc = MockDatabase.getDocument(documentId);
        return doc.toString();
    }
```

```
    public static String checkMedicalNecessity(String diagnosis, String
procedures) {
        boolean isNecessary = MockDatabase.checkMedicalRule(diagnosis,
procedures);
        return String.format("{\"diagnosis\": \"%s\", \"procedures\": \"%s\",
\"isMedicalNecessary\": %b}",
                diagnosis, procedures, isNecessary);
    }
}
```

3. Trình điều khiển Agent chạy với Ollama (ClaimAgentRunner.java)

File này sử dụng HttpClient có sẵn trong Java để gửi yêu cầu tới Ollama local trên máy Mac M2 (http://localhost:11434/v1/chat/completions).

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClaimAgentRunner {

    private static final String OLLAMA_URL =
"http://localhost:11434/v1/chat/completions";

    public static void main(String[] args) {

System.out.println("================================================");
        System.out.println("🚀 BẮT ĐẦU CHẠY CLAIM ASSESSMENT AGENT (JAVA
THUẦN)");

System.out.println("================================================");

        // Mô phỏng gọi tool trực tiếp bằng Java thuần để kiểm tra logic kết
nối
        printToolExecution("verifyDocument",
AgentTools.verifyDocument("DOC-101"));
        printToolExecution("lookupPolicy", AgentTools.lookupPolicy("POL-001"));
        printToolExecution("checkMedicalNecessity",
AgentTools.checkMedicalNecessity("Acute Appendicitis", "Appendectomy"));
        printToolExecution("calculateBenefit",
AgentTools.calculateBenefit("POL-001", "Inpatient", 15000000.0));

        // Kiểm tra kết nối tới Ollama local
        testOllamaConnection();
    }

    private static void printToolExecution(String toolName, String result) {
        System.out.println("\n🛠️ [TOOL EXECUTION]: " + toolName);
        System.out.println("📤 [OUTPUT]: " + result);
```

```
    }

    private static void testOllamaConnection() {
        try {
            HttpClient client = HttpClient.newHttpClient();
            String jsonPayload = """
                {
                  "model": "qwen2.5-coder:7b",
                  "messages": [
                    {"role": "system", "content": "You are a helpful claim
assessment AI assistant."},
                    {"role": "user", "content": "Xin chào, hãy đánh giá hồ sơ
bảo hiểm mã POL-001 cho tôi."}
                  ],
                  "stream": false
                }
                """;

            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create(OLLAMA_URL))
                    .header("Content-Type", "application/json")
                    .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                    .build();

            System.out.println("\n⏳ Đang kiểm tra kết nối tới Ollama
local...");
            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

            if (response.statusCode() == 200) {
                System.out.println("✅ Kết nối Ollama thành công!");
                System.out.println("📥 Phản hồi từ mô hình Qwen:");
                System.out.println(response.body());
            } else {
                System.out.println("❌ Lỗi phản hồi từ Ollama, Status code: " +
response.statusCode());
            }

        } catch (Exception e) {
            System.out.println("❌ Không thể kết nối tới Ollama. Hãy đảm bảo
Ollama đang chạy trên máy (http://localhost:11434).");
            e.printStackTrace();
        }
    }
}
```

Hướng dẫn chạy trên Mac M2:

1. Đảm bảo ứng dụng **Ollama** đang bật trên máy Mac của bạn.
2. Lưu 3 đoạn code trên vào 3 file tương ứng (MockDatabase.java, AgentTools.java, ClaimAgentRunner.java) trong cùng một thư mục.

3. Mở Terminal, di chuyển đến thư mục chứa các file và biên dịch/chạy trực tiếp bằng Java (Java 11+ hỗ trợ chạy file đơn lẻ):
```
javac MockDatabase.java AgentTools.java ClaimAgentRunner.java
java ClaimAgentRunner
```

Cách tiếp cận này giúp bạn kiểm soát toàn bộ logic bằng Java thuần, dễ dàng mở rộng và tích hợp vào hệ thống enterprise sau này mà không phụ thuộc vào các framework phức tạp!

User prompt: Kết quả này là sao? Agent Ollama đóng vai trò gì?

Response: Nhìn vào hình ảnh kết quả chạy chương trình Java của bạn, chúng ta có thể phân tích chi tiết như sau:

1. Kết quả này là sao?

- **Phần 1 (In ra 4 Tool Executions trên cùng):** Trong đoạn code mẫu Java vừa rồi, chương trình đang gọi trực tiếp tuần tự 4 hàm công cụ (verifyDocument, lookupPolicy, checkMedicalNecessity, calculateBenefit) để mô phỏng và kiểm tra xem dữ liệu giả lập trả về có đúng định dạng hay không. Tất cả đều chạy thành công và trả về kết quả JSON rõ ràng (ví dụ: tính toán số tiền bảo hiểm chi trả finalPayout = 13500000.0).
- **Phần 2 (Kết nối Ollama thành công):** Đoạn code đã gửi một HTTP Request thành công tới mô hình **Qwen2.5-Coder:7b** đang chạy ngầm trên máy Mac M2 của bạn (http://localhost:11434).
- **Phần 3 (Phản hồi từ mô hình Qwen ở dưới cùng):** Mô hình Qwen đã nhận được câu chào và prompt đơn giản từ code Java ("Xin chào, hãy đánh giá hồ sơ bảo hiểm mã POL-001 cho tôi"). Do trong prompt test đó chưa cung cấp đủ chi tiết hồ sơ (như loại hình, chẩn đoán, số tiền), mô hình đã phản hồi lại bằng tiếng Việt, phân tích logic và yêu cầu bổ sung thêm các thông tin chi tiết cần thiết để có thể tiến hành thẩm định chính xác.

2. Agent Ollama đóng vai trò gì trong hệ thống này?

Trong kiến trúc hệ thống xử lý hồ sơ bồi thường bảo hiểm này, **Ollama (chạy mô hình Qwen2.5-Coder)** đóng vai trò là **Bộ não trung tâm (LLM Engine / AI Agent)** với các nhiệm vụ cốt lõi sau:

- **Hiểu ngữ cảnh tự nhiên (Natural Language Understanding):** Đọc hiểu yêu cầu nghiệp vụ phức tạp bằng văn bản tự nhiên do người dùng hoặc hệ thống gửi vào.
- **Lên kế hoạch và điều phối (Reasoning & Orchestration):** Tự động suy luận xem cần gọi công cụ nào (Tools), gọi theo thứ tự nào (ví dụ: kiểm tra tài liệu trước, rồi tra cứu chính sách, kiểm tra y tế, và cuối cùng mới tính toán quyền lợi).
- **Tổng hợp và Ra quyết định (Synthesis & Decision Making):** Nhận dữ liệu đầu ra từ các công cụ (dạng JSON thô từ database) để phân tích, đối chiếu với điều khoản, từ đó đưa ra kết luận cuối cùng (APPROVE, REJECT, hay REQUEST_MORE_INFO) kèm theo trích dẫn điều khoản rõ ràng cho người đánh giá.

Nói cách khác, **Java** đóng vai trò là khung xương (thực thi code, quản lý database, gọi API), còn **Ollama / Qwen** đóng vai trò là "chuyên viên tư vấn AI" đưa ra các quyết định nghiệp vụ thông minh dựa trên dữ liệu mà Java cung cấp.

User prompt: Với yêu cầu của bài tập này thì có đáp ứng không?

Response: Dựa trên tài liệu yêu cầu của **AI Challenge 11** và những gì chúng ta vừa xây dựng, câu trả lời ngắn gọn là: **Hiện tại chúng ta đã xây dựng được "khung xương" và các công cụ nền tảng rất chuẩn xác, nhưng để đáp ứng 100% đầy đủ tiêu chí của đề bài thì vẫn còn một vài bước cần hoàn thiện thêm.**

Dưới đây là bảng đánh giá chi tiết xem chúng ta đã làm được gì và còn thiếu những gì so với yêu cầu của đề bài:

1. Những phần đã đáp ứng tốt (✅ Đã hoàn thành)

- **Hệ thống 4 Tools bắt buộc**: Đã lập trình thành công các hàm đúng tên và chức năng (lookupPolicy, calculateBenefit, verifyDocument, checkMedicalNecessity).
- **Môi trường chạy cục bộ (Ollama & Qwen2.5-Coder)**: Đã thiết lập thành công trên máy Mac M2, kết nối qua Java thuần hoặc Python, chạy mượt mà không tốn phí.
- **Logging/Traceability**: Các tool call, input, và output đều được in ra màn hình rõ ràng, đáp ứng tiêu chí ghi lại lịch sử gọi công cụ (log).

2. Những phần chưa đáp ứng hoặc cần bổ sung thêm (⚠️ Cần hoàn thiện)

- **3 Test Cases hoàn chỉnh**: Đề bài yêu cầu phải xây dựng đủ **3 kịch bản test cụ thể** (Phê duyệt, Từ chối, Yêu cầu bổ sung) kèm theo dữ liệu chi tiết về policy, claim, và expected outcome. Hiện tại chúng ta mới chỉ dựng mock data đơn lẻ cho 1 trường hợp.
- **Cấu trúc Báo cáo đánh giá 6 phần (Assessment Report)**: Đề bài yêu cầu báo cáo đầu ra phải có đủ 6 phần cấu trúc rõ ràng: *Document Review, Policy Verification, Medical Necessity, Benefit Calculation, Recommendation, Policy Citations*. Hiện tại mô hình Qwen mới chỉ ở bước phản hồi dạng văn bản chung chung, chưa được ép buộc xuất ra đúng khuôn dạng 6 phần này.
- **Vòng lặp Agent thông minh tự động hoàn toàn (Autonomous Agent Loop)**: Với code Java thuần ở bước trước, chúng ta mới chỉ gọi thủ công các tool. Để đạt chuẩn AI Agent của thử thách, code cần cho phép mô hình AI tự quyết định gọi tool nào, nhận kết quả và tự sinh báo cáo cuối cùng trong một chu trình khép kín.

3. Đề xuất các bước tiếp theo để hoàn thành 100% thử thách:

1. **Hoàn thiện bộ 3 Test Cases**: Viết đủ dữ liệu cho 3 kịch bản (Case 1: Hợp lệ → Approve; Case 2: Vượt hạn mức/loại trừ → Reject; Case 3: Thiếu giấy tờ → Request More Info).
2. **Củng cố System Prompt**: Đưa cấu trúc báo cáo 6 phần vào System Prompt để ép mô hình Qwen xuất ra đúng định dạng chuẩn.
3. **Đóng gói dự án**: Đưa toàn bộ mã nguồn lên GitHub repository kèm theo tài liệu mô tả (Writeup) về thiết kế prompt và tool theo đúng yêu cầu nộp bài của đề.

User prompt: Bạn làm giúp tôi được không?

Response: Hoàn toàn được chứ! Tôi rất sẵn lòng hỗ trợ bạn hoàn thành toàn bộ thử thách **AI Challenge 11 — Claim Assessment AI Agent** này từ đầu đến cuối một cách chỉn chu nhất.

Để hoàn thành bài nộp đạt chuẩn và sẵn sàng đưa lên GitHub theo đúng yêu cầu của đề bài, chúng ta sẽ

cùng nhau thực hiện các bước trọng tâm sau:

1. **Xây dựng 3 Test Cases hoàn chỉnh**: Tạo dữ liệu chi tiết cho 3 kịch bản: phê duyệt trực tiếp, từ chối (do vượt hạn mức hoặc điều khoản loại trừ), và yêu cầu bổ sung thông tin (do thiếu/chưa hoàn thiện tài liệu).
2. **Hoàn thiện mã nguồn (bằng Java hoặc Python)**: Đảm bảo Agent gọi đúng 4 công cụ (verifyDocument, lookupPolicy, checkMedicalNecessity, calculateBenefit) theo đúng trình tự logic và có ghi lại log đầy đủ.
3. **Chuẩn hóa Báo cáo đánh giá (Assessment Report)**: Cấu hình System Prompt để mô hình AI tự động sinh ra báo cáo đủ 6 phần (Document Review, Policy Verification, Medical Necessity, Benefit Calculation, Recommendation, Policy Citations) với các trích dẫn điều khoản cụ thể.
4. **Soạn thảo tài liệu Writeup**: Viết phần giải thích ngắn gọn về quyết định thiết kế System Prompt và Tool Design để đính kèm trong repository GitHub.

Bạn muốn chúng ta tiếp tục triển khai bằng **Java** hay chuyển sang **Python**, và có muốn bắt đầu bằng việc xây dựng **3 bộ Test Cases** ngay bây giờ không?

User prompt: Tôi dùng Java bạn làm giúp tôi từng bước

Response: Tuyệt vời! Chúng ta sẽ triển khai từng bước toàn bộ dự án **AI Challenge 11** bằng **Java thuần** (Core Java) để chạy mượt mà với Ollama trên máy Mac M2 của bạn.

Theo yêu cầu của đề bài, chúng ta cần hoàn thành các phần cốt lõi sau:

1. **3 Test Cases hoàn chỉnh** (Phê duyệt, Từ chối do vượt hạn mức, và Yêu cầu bổ sung do thiếu tài liệu).
2. **4 Công cụ (Tools)** (verifyDocument, lookupPolicy, checkMedicalNecessity, calculateBenefit).
3. **Hệ thống Agent & Báo cáo đánh giá chuẩn 6 phần** có trích dẫn điều khoản và ghi log lịch sử gọi tool.

BƯỚC 1: Xây dựng cơ sở dữ liệu giả lập và 3 Test Cases (MockDatabase.java)

Bạn hãy tạo file MockDatabase.java. File này chứa toàn bộ dữ liệu chính sách, trạng thái tài liệu, các quy tắc y tế và định nghĩa sẵn **3 Test Cases** theo đúng chuẩn đề bài.

```
import java.util.*;

public class MockDatabase {

    // 1. Kho dữ liệu Chính sách bảo hiểm (Policies)
    public static Map<String, Object> getPolicy(String policyId) {
        if ("POL-001".equals(policyId)) {
            Map<String, Object> policy = new HashMap<>();
            policy.put("policyId", "POL-001");
            policy.put("holderName", "Nguyen Van A");
            policy.put("status", "ACTIVE");
            policy.put("effectiveDate", "2025-01-01");
            policy.put("expiryDate", "2026-12-31");
```

```
            Map<String, Map<String, Object>> benefits = new HashMap<>();
            benefits.put("Inpatient", Map.of("limit", 50000000.0, "copay", 0.1,
"waitingPeriodDays", 30));
            benefits.put("Outpatient", Map.of("limit", 10000000.0, "copay",
0.2, "waitingPeriodDays", 0));
            policy.put("benefits", benefits);

            policy.put("exclusions", List.of("Cosmetic surgery", "Experimental
treatments"));
            return policy;
        }
        return null;
    }

    // 2. Kho dữ liệu Tài liệu (Documents)
    public static Map<String, Object> getDocument(String documentId) {
        if ("DOC-101".equals(documentId)) {
            return Map.of(
                "documentId", "DOC-101",
                "type", "Medical Bill / Invoice",
                "status", "complete",
                "issues", List.of()
            );
        } else if ("DOC-102".equals(documentId)) {
            return Map.of(
                "documentId", "DOC-102",
                "type", "Discharge Summary",
                "status", "incomplete",
                "issues", List.of("Missing physician signature and official
hospital stamp")
            );
        }
        return Map.of("error", "Document not found", "status", "missing");
    }

    // 3. Quy tắc y tế (Medical Necessity Rules)
    public static boolean checkMedicalRule(String diagnosis, String procedures)
{
        return "Acute Appendicitis".equals(diagnosis) &&
"Appendectomy".equals(procedures);
    }

    // 4. Định nghĩa 3 Test Cases theo yêu cầu đề bài
    public static List> getTestCases() {
        List> cases = new ArrayList<>();

        // Case 1: Straightforward Approval (Phê duyệt trực tiếp)
        cases.add(Map.of(
            "caseId", "CASE-1",
            "description", "Straightforward Approval",
            "policyId", "POL-001",
            "claimType", "Inpatient",
```

```
            "amount", "15000000",
            "diagnosis", "Acute Appendicitis",
            "procedures", "Appendectomy",
            "documentId", "DOC-101",
            "expectedOutcome", "APPROVE"
        ));

        // Case 2: Rejection - Limit Exceeded (Từ chối do vượt hạn mức 50tr)
        cases.add(Map.of(
            "caseId", "CASE-2",
            "description", "Rejection - Limit Exceeded",
            "policyId", "POL-001",
            "claimType", "Inpatient",
            "amount", "80000000",
            "diagnosis", "Acute Appendicitis",
            "procedures", "Appendectomy",
            "documentId", "DOC-101",
            "expectedOutcome", "REJECT"
        ));

        // Case 3: Request More Info - Incomplete Document (Yêu cầu bổ sung do thiếu chữ ký/dấu)
        cases.add(Map.of(
            "caseId", "CASE-3",
            "description", "Request More Info due to incomplete doc",
            "policyId", "POL-001",
            "claimType", "Inpatient",
            "amount", "15000000",
            "diagnosis", "Acute Appendicitis",
            "procedures", "Appendectomy",
            "documentId", "DOC-102",
            "expectedOutcome", "REQUEST_MORE_INFO"
        ));

        return cases;
    }
}
```

Hướng dẫn thực hiện bước này:

1. Bạn tạo file MockDatabase.java trong thư mục dự án và dán đoạn code trên vào.
2. Hãy cho tôi biết khi bạn đã chuẩn bị xong file này để chúng ta chuyển sang **Bước 2: Viết 4 hàm công cụ (AgentTools.java)** nhé!

User prompt: sang bước 2 giúp tôi

Response: Tuyệt vời! Chúng ta sẽ tiếp tục với **Bước 2: Viết 4 hàm công cụ (Tools)** theo đúng yêu cầu của đề bài AI Challenge 11 (lookupPolicy, calculateBenefit, verifyDocument, checkMedicalNecessity).

BƯỚC 2: Xây dựng file chứa 4 Công cụ (AgentTools.java)

Bạn hãy tạo file AgentTools.java trong cùng thư mục dự án và dán đoạn mã sau vào. Các hàm này sẽ gọi dữ liệu từ MockDatabase và trả về kết quả định dạng JSON giả lập để chuẩn bị cho Agent xử lý:

```
import java.util.Map;

public class AgentTools {

    // 1. lookupPolicy(policyId) — Tra cứu điều khoản, hạn mức, loại trừ[cite: 1, 7]
    public static String lookupPolicy(String policyId) {
        Map<String, Object> policy = MockDatabase.getPolicy(policyId);
        if (policy == null) {
            return "{\"error\": \"Policy ID " + policyId + " not found.\"}";
        }
        return policy.toString();
    }

    // 2. calculateBenefit(policyId, claimType, amount) — Tính toán số tiền chi trả, copay, hạn mức[cite: 1, 7]
    public static String calculateBenefit(String policyId, String claimType, double amount) {
        Map<String, Object> policy = MockDatabase.getPolicy(policyId);
        if (policy == null) {
            return "{\"error\": \"Policy ID " + policyId + " not found.\"}";
        }

        @SuppressWarnings("unchecked")
        Map<String, Map<String, Object>> benefits = (Map<String, Map<String, Object>>) policy.get("benefits");
        Map<String, Object> benefitInfo = benefits != null ? benefits.get(claimType) : null;

        if (benefitInfo == null) {
            return "{\"error\": \"Claim type " + claimType + " not covered under this policy.\"}";
        }

        double limit = (double) benefitInfo.get("limit");
        double copayRate = (double) benefitInfo.get("copay");

        double coveredBeforeCopay = Math.min(amount, limit);
        double copayAmount = coveredBeforeCopay * copayRate;
        double finalPayout = coveredBeforeCopay - copayAmount;
        double remainingLimit = (amount <= limit) ? (limit - amount) : 0.0;

        return String.format(
            "{\"submittedAmount\": %.1f, \"coveredAmount\": %.1f, \"copayAmount\": %.1f, \"finalPayout\": %.1f, \"remainingLimit\": %.1f, \"exceededLimit\": %b}",
```

```
            amount, coveredBeforeCopay, copayAmount, finalPayout,
remainingLimit, (amount > limit)
        );
    }

    // 3. verifyDocument(documentId) — Kiểm tra loại tài liệu, trạng thái
completeness và lỗi[cite: 1, 7]
    public static String verifyDocument(String documentId) {
        Map<String, Object> doc = MockDatabase.getDocument(documentId);
        return doc.toString();
    }

    // 4. checkMedicalNecessity(diagnosis, procedures) — Kiểm tra tính phù hợp
y tế [cite: 1, 7]
    public static String checkMedicalNecessity(String diagnosis, String
procedures) {
        boolean isNecessary = MockDatabase.checkMedicalRule(diagnosis,
procedures);
        return String.format(
            "{\"diagnosis\": \"%s\", \"procedures\": \"%s\",
\"isMedicalNecessary\": %b, \"note\": \"%s\"}",
            diagnosis, procedures, isNecessary,
            isNecessary ? "Clinically appropriate" : "Treatment does not match
diagnosis or lacks clinical justification."
        );
    }
}
```

Khi bạn đã tạo xong file AgentTools.java, hãy phản hồi lại cho tôi để chúng ta chuyển sang **Bước 3: Xây dựng trình điều khiển Agent (ClaimAgentRunner.java)** tích hợp với Ollama & Qwen để chạy toàn bộ 3 Test Cases và xuất báo cáo chuẩn 6 phần nhé!

User prompt: tôi đã xong bước 2

Response: Rất tuyệt vời! Bây giờ chúng ta sẽ chuyển sang **Bước 3: Xây dựng trình điều khiển Agent (ClaimAgentRunner.java)**.

File này sẽ đóng vai trò tự động duyệt qua **3 Test Cases** đã định nghĩa, gọi lần lượt các công cụ (verifyDocument, lookupPolicy, checkMedicalNecessity, calculateBenefit), ghi lại log lịch sử thực thi (traceability) và yêu cầu mô hình Qwen trên Ollama tổng hợp thành **Báo cáo đánh giá chuẩn 6 phần** theo đúng yêu cầu của đề bài.

BƯỚC 3: Xây dựng file chạy chính (ClaimAgentRunner.java)

Bạn hãy tạo file ClaimAgentRunner.java trong cùng thư mục dự án và dán đoạn mã sau vào:

```
import java.net.URI;
```

```
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.*;

public class ClaimAgentRunner {

    private static final String OLLAMA_URL =
"http://localhost:11434/v1/chat/completions";

    public static void main(String[] args) {

System.out.println("====================================================");
        System.out.println("🚀 BẮT ĐẦU CHẠY CLAIM ASSESSMENT AGENT (3 TEST
CASES)");

System.out.println("====================================================");

        List> testCases = MockDatabase.getTestCases();

        for (Map<String, String> testCase : testCases) {
            runSingleClaimAssessment(testCase);

System.out.println("\n----------------------------------------------------\n");
        }
    }

    private static void runSingleClaimAssessment(Map<String, String> tc) {
        String caseId = tc.get("caseId");
        String description = tc.get("description");
        String policyId = tc.get("policyId");
        String claimType = tc.get("claimType");
        double amount = Double.parseDouble(tc.get("amount"));
        String diagnosis = tc.get("diagnosis");
        String procedures = tc.get("procedures");
        String documentId = tc.get("documentId");

        System.out.println("📌 Đang xử lý: " + caseId + " - " + description);

        // 1. Thực thi các công cụ theo đúng trình tự logic bắt buộc
        System.out.println("  -> [Step 1] Xác minh tài liệu...");
        String docResult = AgentTools.verifyDocument(documentId);
        System.out.println("     Output: " + docResult);

        System.out.println("  -> [Step 2] Tra cứu chính sách bảo hiểm...");
        String policyResult = AgentTools.lookupPolicy(policyId);
        System.out.println("     Output: [Policy Data Retrieved]");

        System.out.println("  -> [Step 3] Kiểm tra tính cần thiết y tế...");
        String medicalResult = AgentTools.checkMedicalNecessity(diagnosis,
procedures);
        System.out.println("     Output: " + medicalResult);
```

```
        System.out.println("  -> [Step 4] Tính toán quyền lợi bảo hiểm...");
        String benefitResult = AgentTools.calculateBenefit(policyId, claimType,
amount);
        System.out.println("     Output: " + benefitResult);

        // 2. Chuẩn bị Prompt tổng hợp báo cáo gửi cho Ollama (Qwen)
        String userPrompt = String.format(
            "Hãy đóng vai trò là Claim Assessment AI Agent. Dựa vào kết quả các
công cụ dưới đây, hãy lập Báo cáo đánh giá gồm đúng 6 phần: " +
            "1. Document Review, 2. Policy Verification, 3. Medical Necessity,
4. Benefit Calculation, 5. Recommendation (APPROVE / REJECT /
REQUEST_MORE_INFO), 6. Policy Citations.\n\n" +
            "Dữ liệu đầu vào:\n- Document Result: %s\n- Policy Result: %s\n-
Medical Result: %s\n- Benefit Result: %s\n" +
            "Lưu ý quy tắc: Nếu tài liệu thiếu/incomplete, phải đề xuất
REQUEST_MORE_INFO, không được từ chối.",
            docResult, policyResult, medicalResult, benefitResult
        );

        // 3. Gửi yêu cầu tới Ollama local trên máy Mac M2
        callOllamaAgent(userPrompt);
    }

    private static void callOllamaAgent(String prompt) {
        try {
            HttpClient client = HttpClient.newHttpClient();

            // Xây dựng JSON payload an toàn (Escape dấu ngoặc kép cơ bản)
            String escapedPrompt = prompt.replace("\"", "\\\"").replace("\n",
"\\n");
            String jsonPayload = String.format("""
                {
                  "model": "qwen2.5-coder:7b",
                  "messages": [
                    {"role": "system", "content": "You are an expert insurance
claim assessment AI agent that produces structured 6-section reports with exact
policy citations."},
                    {"role": "user", "content": "%s"}
                  ],
                  "stream": false
                }
                """, escapedPrompt);

            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create(OLLAMA_URL))
                    .header("Content-Type", "application/json")
                    .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                    .build();

            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
```

```
            if (response.statusCode() == 200) {
                System.out.println("\n📋 [BÁO CÁO ĐÁNH GIÁ TỪ AGENT]:");
                // In ra nội dung phản hồi từ mô hình Qwen
                String body = response.body();
                int contentIndex = body.indexOf("\"content\":\"");
                if (contentIndex != -1) {
                    int startIndex = contentIndex + 11;
                    int endIndex = body.indexOf("\"+\"}", startIndex); // Tùy
chọn, trích xuất text thô đơn giản
                    // In gọn nội dung phản hồi
                    System.out.println(body.substring(startIndex,
Math.min(startIndex + 1500, body.length()))
                            .replace("\\n", "\n").replace("\\\"", "\""));
                } else {
                    System.out.println(body);
                }
            } else {
                System.out.println("❌ Lỗi gọi Ollama API, mã lỗi: " +
response.statusCode());
            }

        } catch (Exception e) {
            System.out.println("❌ Không thể kết nối tới Ollama. Hãy đảm bảo
Ollama đang chạy trên máy.");
            e.printStackTrace();
        }
    }
}
```

Cách biên dịch và chạy thử toàn bộ hệ thống trên Terminal (Mac M2):

Sau khi bạn đã có đủ 3 file (MockDatabase.java, AgentTools.java, ClaimAgentRunner.java) trong cùng thư mục dự án, hãy mở Terminal và chạy các lệnh sau:

```
# 1. Biên dịch tất cả các file Java
javac MockDatabase.java AgentTools.java ClaimAgentRunner.java

# 2. Chạy chương trình
java ClaimAgentRunner
```

Bạn hãy thử chạy và cho tôi biết kết quả hiển thị trên màn hình nhé!

User prompt: Thêm code để in ra step call đến Ollama

Response: Để in ra chi tiết bước gọi (step call) đến Ollama — bao gồm cả Endpoint URL và toàn bộ JSON Payload (system prompt, user prompt và cấu hình model) trước khi gửi request — chúng ta có thể bổ sung

các câu lệnh System.out.println ngay bên trong hàm callOllamaAgent của file ClaimAgentRunner.java.

Dưới đây là phiên bản cập nhật của hàm callOllamaAgent để log lại toàn bộ thông tin step call:

Cập nhật hàm callOllamaAgent trong ClaimAgentRunner.java

Bạn hãy mở file ClaimAgentRunner.java và thay thế phương thức callOllamaAgent cũ bằng đoạn mã sau:

```
    private static void callOllamaAgent(String prompt) {
        try {
            HttpClient client = HttpClient.newHttpClient();

            // Xây dựng JSON payload an toàn
            String escapedPrompt = prompt.replace("\"", "\\\"").replace("\n",
"\\n");
            String jsonPayload = String.format("""
                {
                  "model": "qwen2.5-coder:7b",
                  "messages": [
                    {"role": "system", "content": "You are an expert insurance
claim assessment AI agent that produces structured 6-section reports with exact
policy citations."},
                    {"role": "user", "content": "%s"}
                  ],
                  "stream": false
                }
                """, escapedPrompt);

            // ============================================================
            // 🔍 IN RA STEP CALL CHI TIẾT GỬI ĐẾN OLLAMA
            // ============================================================

System.out.println("\n----------------------------------------------------bytes")
;
            System.out.println("📤 [OLLAMA STEP CALL - REQUEST DETAILS]");
            System.out.println(" 🔹 Target URL : " + OLLAMA_URL);
            System.out.println(" 🔹 Model Name : qwen2.5-coder:7b");
            System.out.println(" 🔹 Payload Body (JSON):\n" + jsonPayload);

System.out.println("------------------------------------------------");

            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create(OLLAMA_URL))
                    .header("Content-Type", "application/json")
                    .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                    .build();

            System.out.println("⏳ Đang gửi request tới Ollama local trên Mac
M2...");
            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
```

```
            if (response.statusCode() == 200) {
                System.out.println("\n📥 [OLLAMA RESPONSE - SUCCESS]:");
                String body = response.body();

                // Trích xuất nội dung 'content' trả về từ JSON của Ollama để
hiển thị gọn gàng
                int contentIndex = body.indexOf("\"content\":\"");
                if (contentIndex != -1) {
                    int startIndex = contentIndex + 11;
                    // Tìm vị trí kết thúc nội dung content (trước field tiếp
theo hoặc cuối chuỗi)
                    System.out.println(body.substring(startIndex, body.length()
- 3)
                            .replace("\\n", "\n")
                            .replace("\\\"", "\""));
                } else {
                    System.out.println(body);
                }
            } else {
                System.out.println("❌ Lỗi gọi Ollama API, mã trạng thái: " +
response.statusCode());
                System.out.println("Chi tiết lỗi: " + response.body());
            }

        } catch (Exception e) {
            System.out.println("❌ Không thể kết nối tới Ollama. Hãy đảm bảo
Ollama đang chạy trên máy (http://localhost:11434).");
            e.printStackTrace();
        }
    }
```

Hướng dẫn kiểm tra:

Sau khi cập nhật code vào file ClaimAgentRunner.java, bạn dịch và chạy lại:

```
javac MockDatabase.java AgentTools.java ClaimAgentRunner.java
java ClaimAgentRunner
```

Khi chạy, trên màn hình Terminal sẽ hiển thị rõ ràng từng bước:

1. **Tool Execution logs**: Kết quả chạy của 4 công cụ (verifyDocument, lookupPolicy, checkMedicalNecessity, calculateBenefit).
2. **Ollama Step Call Request**: In ra chính xác URL và toàn bộ payload JSON được truyền vào mô hình Qwen.
3. **Ollama Response**: Phản hồi được trả về từ tiến trình AI cục bộ trên MacBook M2 của bạn.

User prompt: nếu thay bằng API GEMINI google thì thay bằng đoạn nào?

Response: Để chuyển từ **Ollama (chạy local)** sang sử dụng **Google Gemini API** trong file ClaimAgentRunner.java, bạn có thể tận dụng endpoint tương thích OpenAI (OpenAI-compatible endpoint) mà Google cung cấp. Nhờ đó, bạn **không cần thay đổi cấu trúc JSON payload**, chỉ cần điều chỉnh lại **URL**, **API Key** và **Tên Model**.

Theo tài liệu của đề bài, bạn hoàn toàn có thể sử dụng Gemini API thông qua key miễn phí từ Google AI Studio.

Dưới đây là các phần cần thay đổi cụ thể trong file ClaimAgentRunner.java:

Các bước thay đổi trong ClaimAgentRunner.java:

1. Thay đổi URL và Thêm API Key ở đầu file

Thay vì trỏ vào localhost:11434, bạn đổi sang endpoint của Gemini và thêm API Key của bạn vào:

```
    // 🌐 Đổi URL sang Google Gemini OpenAI-compatible endpoint
    private static final String GEMINI_URL =
"https://generativelanguage.googleapis.com/v1beta/openai/chat/completions";

    // 🔑 Nhập Google Gemini API Key của bạn vào đây (Lấy miễn phí tại
https://aistudio.google.com/)
    private static final String GEMINI_API_KEY = "AIzaSyYourGeminiApiKeyHere";
```

2. Cập nhật phương thức gọi API (callGeminiAgent)

Bạn thay thế toàn bộ phương thức gọi API cũ bằng đoạn code sau. Điểm khác biệt là đổi tên model thành "gemini-1.5-flash" (hoặc "gemini-1.5-pro") và bổ sung Header Authorization: Bearer <API_KEY>:

```
    private static void callGeminiAgent(String prompt) {
        try {
            HttpClient client = HttpClient.newHttpClient();

            // Xây dựng JSON payload (giữ nguyên cấu trúc chuẩn OpenAI)
            String escapedPrompt = prompt.replace("\"", "\\\"").replace("\n",
"\\n");
            String jsonPayload = String.format("""
                {
                  "model": "gemini-1.5-flash",
                  "messages": [
                    {"role": "system", "content": "You are an expert insurance
claim assessment AI agent that produces structured 6-section reports with exact
policy citations."},
                    {"role": "user", "content": "%s"}
                  ],
                  "temperature": 0.2
```

```
                }
                """, escapedPrompt);


System.out.println("\n----------------------------------------------------");
            System.out.println("📤 [GEMINI STEP CALL - REQUEST DETAILS]");
            System.out.println(" 🔹 Target URL : " + GEMINI_URL);
            System.out.println(" 🔹 Model Name : gemini-1.5-flash");

System.out.println("----------------------------------------------------");

            // Xây dựng HttpRequest kèm Header Authorization chứa Gemini API
Key
            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create(GEMINI_URL))
                    .header("Content-Type", "application/json")
                    .header("Authorization", "Bearer " + GEMINI_API_KEY)
                    .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
                    .build();

            System.out.println("⏳ Đang gửi request tới Google Gemini API...");
            HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

            if (response.statusCode() == 200) {
                System.out.println("\n📥 [GEMINI RESPONSE - SUCCESS]:");
                String body = response.body();

                // Trích xuất nội dung 'content' trả về từ JSON
                int contentIndex = body.indexOf("\"content\":\"");
                if (contentIndex != -1) {
                    int startIndex = contentIndex + 11;
                    // In ra nội dung phản hồi từ Gemini
                    System.out.println(body.substring(startIndex, body.length()
- 3)
                            .replace("\\n", "\n")
                            .replace("\\\"", "\""));
                } else {
                    System.out.println(body);
                }
            } else {
                System.out.println("❌ Lỗi gọi Gemini API, mã trạng thái: " +
response.statusCode());
                System.out.println("Chi tiết lỗi: " + response.body());
            }

        } catch (Exception e) {
            System.out.println("❌ Không thể kết nối tới Google Gemini API.");
            e.printStackTrace();
        }
    }
```

3. Cập nhật lời gọi trong hàm main

Đừng quên trong vòng lặp xử lý của hàm main, thay vì gọi callOllamaAgent(userPrompt);, bạn hãy gọi sang hàm mới:

```
// Gọi qua Gemini API thay vì Ollama
callGeminiAgent(userPrompt);
```

Tóm tắt các điểm mấu chốt khi đổi sang Gemini:

1. **Endpoint**: https://generativelanguage.googleapis.com/v1beta/openai/chat/completions
2. **Authentication**: Thêm .header("Authorization", "Bearer " + GEMINI_API_KEY) vào HttpRequest.
3. **Model Name**: Thay vì "qwen2.5-coder:7b", hãy dùng "gemini-1.5-flash" hoặc "gemini-1.5-pro".
